Buổi 5

Sắp xếp trộn và sắp xếp ngoại

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Sắp xếp bằng phương pháp trộn trực tiếp

1.1 Thao tác trộn hai dãy đã có thứ tự
1.2 Trộn trực tiếp
1.3 Straight Merge Sort – cài đặt
1.4 Straight Merge Sort – minh họa
1.5 Phân tích sắp xếp trộn

2. Trộn tự nhiên (Natural Merge Sort)

2.1 Trộn tự nhiên – tư tưởng và minh họa

3. Sắp xếp ngoại

3.1 Sắp xếp ngoại – đặt vấn đề
3.2 Trộn trực tiếp trên tập tin
3.3 Cài đặt: phân phối và chương trình chính
3.4 Cài đặt: trộn hai tập tin theo từng cặp run
3.5 Cải tiến sắp xếp ngoại: run ban đầu và trộn nhiều đường

4. Sắp xếp theo chỉ mục

4.1 Sắp xếp theo chỉ mục – tư tưởng và cài đặt
4.2 Sắp xếp theo chỉ mục – ví dụ và đánh giá

5. Tổng kết và thực hành

5.1 Bảng so sánh các thuật toán sắp xếp
5.2 Bài thực hành buổi 5
5.3 Câu hỏi và bài tập

1.1 Thao tác trộn hai dãy đã có thứ tự

- **Bài toán trộn (merge):** cho hai dãy **đã có thứ tự** A (Na phần tử) và B (Nb phần tử), tạo dãy C có thứ tự gồm mọi phần tử của A và B.
- **Ý tưởng:** so sánh hai phần tử **đầu** của A và B, đưa phần tử **nhỏ hơn** sang C; khi một dãy hết, chép phần còn lại của dãy kia.

```
void Merge(T A[], int Na, T B[], int Nb, T C[])
{  int i = 0, j = 0, k = 0;
   while (i < Na && j < Nb)
      if (A[i] <= B[j])        // <= : giữ ổn định
         C[k++] = A[i++];
      else
         C[k++] = B[j++];
   while (i < Na) C[k++] = A[i++];
   while (j < Nb) C[k++] = B[j++];
}
```

- **Số phép so sánh** ≤ Na + Nb − 1; **số phép gán** = Na + Nb ⇒ trộn là **O(Na + Nb)**.

Ví dụ 5.1 – A = {11, 23, 42, 74}, B = {36, 58, 65, 94}

Bước	A[i]	B[j]	Đưa vào C	C
1	11	36	11	11
2	23	36	23	11 23
3	42	36	36	11 23 36
4	42	58	42	… 36 42
5	74	58	58	… 42 58
6	74	65	65	… 58 65
7	74	94	74	… 65 74
–	hết A		chép 94	11 23 36 42 58 65 74 **94**

- 7 phép so sánh = Na + Nb − 1 (trường hợp xấu nhất).

1.2 Trộn trực tiếp (Straight Merge Sort) – tư tưởng

- **Run (đường chạy):** một đoạn con **có thứ tự** của dãy. Một phần tử đứng riêng là run độ dài 1.
- **Tư tưởng (giáo trình – trộn trực tiếp):** xem dãy gồm các run độ dài **k = 1**. Lặp:
  - **Phân phối** luân phiên từng run độ dài k của M sang hai dãy phụ **Temp1, Temp2**.
  - **Trộn** từng cặp run tương ứng của Temp1 và Temp2 thành run độ dài **2k**, đặt lại vào M.
  - **k = 2k**; dừng khi k ≥ N.

```
B1: k = 1
B2: IF (k ≥ N) Thực hiện Bkt
B3: Phân phối luân phiên mỗi k phần tử của M
    vào Temp1, Temp2 (N1, N2 phần tử)
B4: Trộn từng cặp run độ dài k
    của Temp1, Temp2 thành run độ dài 2k → M
B5: k = k * 2
B6: Lặp lại B2
Bkt: Kết thúc
```

- Số **lần lặp** (pass) = $\lceil \log_2 N \rceil$ vì độ dài run tăng gấp đôi mỗi lần: 1 → 2 → 4 → … → ≥ N.
- Mỗi pass xử lý mọi phần tử **2 lần** (phân phối + trộn).
- Trộn trực tiếp **không** tận dụng thứ tự sẵn có của dữ liệu (luôn bắt đầu k = 1) ⇒ khắc phục bằng **trộn tự nhiên** (mục 2).
- Cách viết đệ quy "chia đôi – sắp mỗi nửa – trộn" (MergeSort top-down) cho cùng kết quả và độ phức tạp.

1.3 Straight Merge Sort – cài đặt

```
// Trộn run M1[i..i+k-1] và M2[j..j+k-1] vào M từ vị trí p
void MergeRun(T M1[], int N1, int &i,
              T M2[], int N2, int &j,
              T M[], int &p, int k)
{  int e1 = (i + k < N1) ? i + k : N1;
   int e2 = (j + k < N2) ? j + k : N2;
   while (i < e1 && j < e2)
      M[p++] = (M1[i] <= M2[j]) ? M1[i++] : M2[j++];
   while (i < e1) M[p++] = M1[i++];
   while (j < e2) M[p++] = M2[j++];
}
```

```
void StraightMergeSort(T M[], int N)
{  T *Temp1 = new T[N], *Temp2 = new T[N];
   for (int k = 1; k < N; k *= 2)
   {  int N1 = 0, N2 = 0;
      for (int s = 0; s < N; s++)       // phân phối
         if ((s / k) % 2 == 0) Temp1[N1++] = M[s];
         else                  Temp2[N2++] = M[s];
      int i = 0, j = 0, p = 0;
      while (i < N1 || j < N2)          // trộn
         MergeRun(Temp1, N1, i, Temp2, N2, j, M, p, k);
   }
   delete[] Temp1;  delete[] Temp2;
}
```

Lưu ý cài đặt

Bộ nhớ phụ **O(N)** (Temp1, Temp2) – nhược điểm chính của sắp xếp trộn trên mảng. Biến thể đệ quy dùng một mảng phụ N phần tử: `MergeSort(M, L, R)` = MergeSort(M, L, Mid) + MergeSort(M, Mid+1, R) + trộn.

1.4 Straight Merge Sort – minh họa

Ví dụ 5.2 – M = {42, 23, 74, 11, 65, 58, 94, 36}

k	Temp1 (các run)	Temp2 (các run)	M sau khi trộn (run độ dài 2k)
1	42 · 74 · 65 · 94	23 · 11 · 58 · 36	**23 42** · **11 74** · **58 65** · **36 94**
2	23 42 · 58 65	11 74 · 36 94	**11 23 42 74** · **36 58 65 94**
4	11 23 42 74	36 58 65 94	**11 23 36 42 58 65 74 94**
8	k ≥ N ⇒ dừng		

- **Phân tích ví dụ:** 3 pass = $\log_2 8$; mỗi pass phân phối 8 phần tử và trộn 8 phần tử ⇒ 3 × 16 = **48 phép gán**.
- Số phép so sánh: pass k = 1: 4 cặp × 1 = 4; k = 2: 2 cặp × ≤ 3 (thực tế 3 + 3); k = 4: 1 cặp × ≤ 7 (thực tế 7).

Ví dụ 5.3 – N không là lũy thừa của 2: M = {5, 2, 9, 1, 7}

- k = 1: Temp1 = 5 · 9 · 7; Temp2 = 2 · 1 ⇒ M = 2 5 · 1 9 · 7
- k = 2: Temp1 = 2 5 · 7; Temp2 = 1 9 ⇒ M = 1 2 5 9 · 7
- k = 4: Temp1 = 1 2 5 9; Temp2 = 7 ⇒ M = **1 2 5 7 9**
- Run cuối có thể **ngắn hơn** k hoặc **không có cặp** – hàm MergeRun phải xử lý các biên e1, e2.

1.5 Phân tích sắp xếp trộn

Tiêu chí	Giá trị
Số pass	$\lceil \log_2 N \rceil$
Số phép gán mỗi pass	2N (phân phối + trộn)
Số phép so sánh mỗi pass	$\leq N$
Tổng	**O(N log N)** trong **mọi** trường hợp (tốt nhất, xấu nhất, trung bình)
Bộ nhớ phụ	O(N)
Ổn định	**Có** (khi trộn dùng `<=`)

- **Truy hồi** (bản đệ quy): $T(N) = 2T(N/2) + cN \Rightarrow \mathbf{N \log_2 N}$.

Ví dụ 5.4 – $N = 10^6$

$\log_2 10^6 \approx 20$ pass × $2 \cdot 10^6$ phép gán ≈ $4 \cdot 10^7$ phép gán ⇒ cỡ **0,1 giây**; Insertion Sort trung bình ≈ $N^2/4 = 2{,}5 \cdot 10^{11}$ ⇒ cỡ **40 phút**.

- **Ưu điểm:** ổn định; bảo đảm N log N; truy cập dữ liệu **tuần tự** ⇒ rất phù hợp với **danh sách liên kết** và **tập tin** (sắp xếp ngoại).
- **Nhược điểm:** tốn bộ nhớ phụ O(N) khi làm trên mảng; nhiều phép gán hơn QuickSort.

2.1 Trộn tự nhiên – tư tưởng và minh họa

- **Tư tưởng:** tận dụng các **run tự nhiên** sẵn có trong dữ liệu (đoạn tăng dài nhất) thay vì bắt đầu từ run độ dài 1. Mỗi lần lặp: phân phối luân phiên các run tự nhiên sang Temp1, Temp2 rồi trộn từng cặp. Dừng khi M chỉ còn **1 run**.

Ví dụ 5.5 – M = {42, 23, 74, 11, 65, 58, 94, 36}

Lần	Các run tự nhiên của M	Temp1	Temp2	M sau khi trộn
1	[42] [23 74] [11 65] [58 94] [36]	42 · 11 65 · 36	23 74 · 58 94	23 42 74 · 11 58 65 94 · 36
2	[23 42 74] [11 58 65 94] [36]	23 42 74 · 36	11 58 65 94	11 23 42 58 65 74 94 · 36
3	[11 23 42 58 65 74 94] [36]	11 … 94	36	**11 23 36 42 58 65 74 94**
4	Chỉ còn 1 run ⇒ dừng			

Ví dụ 5.6 – Dữ liệu gần có thứ tự: M = {1, 2, 3, 4, 9, 10, 5, 6, 7, 8}

- Run tự nhiên: [1 2 3 4 9 10] [5 6 7 8] ⇒ **chỉ 1 lần trộn** là xong.
- Trộn trực tiếp: vẫn cần $\lceil \log_2 10 \rceil = 4$ pass.

- **Lưu ý:** khi phân phối, hai run kề nhau trong cùng Temp có thể "**dính**" thành một run dài hơn (nếu phần tử cuối run trước ≤ phần tử đầu run sau) ⇒ số run thực tế còn ít hơn.
- **Phân tích:** xấu nhất (dãy giảm, run độ dài 1) như trộn trực tiếp O(N log N); tốt nhất (đã sắp) chỉ 1 lần duyệt **O(N)**.

3.1 Sắp xếp ngoại – đặt vấn đề

- **Bài toán:** sắp xếp tập tin **lớn hơn bộ nhớ trong** (ví dụ 50 GB dữ liệu, RAM 8 GB) ⇒ không thể nạp toàn bộ vào mảng.
- **Đặc điểm của tập tin:** truy cập **tuần tự** nhanh, truy cập ngẫu nhiên (fseek) chậm; chi phí chủ yếu là **số lần đọc/ghi** tập tin (số pass × N).
- ⇒ Phương pháp phù hợp: **trộn** (chỉ cần đọc/ghi tuần tự). Giáo trình trình bày: **trộn trực tiếp**, **trộn tự nhiên** trên tập tin, và **sắp xếp theo chỉ mục**.

Phương pháp nội	Dùng được cho tập tin?	Lý do
QuickSort, HeapSort	Không hiệu quả	Truy cập ngẫu nhiên M[i], M[2i+1] ⇒ fseek liên tục
Bubble, Insertion	Quá chậm	$O(N^2)$ lần đọc/ghi
MergeSort	**Phù hợp**	Đọc/ghi tuần tự, số pass $\log_2 N$

Ví dụ 5.7 – Vì sao không dùng QuickSort trên tập tin?

Tập tin 10^8 bản ghi. QuickSort ≈ $2{,}7 \cdot 10^9$ lần truy cập **ngẫu nhiên**, mỗi lần (đĩa cứng) ≈ 5 ms ⇒ hơn **150 ngày**. Trộn trên tập tin: 27 pass × $2 \cdot 10^8$ bản ghi đọc/ghi **tuần tự** (hàng trăm MB/s) ⇒ cỡ vài chục phút.

3.2 Trộn trực tiếp trên tập tin

- **Tập tin:** Fd (dữ liệu cần sắp), hai tập tin phụ **Ft1, Ft2**.

```
B1: L = 1                          // độ dài run
B2: IF (L ≥ N) Thực hiện Bkt
B3: Phân phối: đọc Fd, ghi luân phiên mỗi
    L phần tử sang Ft1, Ft2
B4: Trộn: đọc từng cặp run độ dài L từ Ft1, Ft2,
    trộn thành run độ dài 2L, ghi lại vào Fd
B5: L = 2 * L
B6: Lặp lại B2
Bkt: Kết thúc
```

- Mỗi pass: đọc N + ghi N (phân phối), đọc N + ghi N (trộn) ⇒ **4N** thao tác vào/ra.

Ví dụ 5.8 – Fd = 42 23 74 11 65 58 94 36

L	Ft1	Ft2	Fd sau trộn
1	42 74 65 94	23 11 58 36	23 42 11 74 58 65 36 94
2	23 42 58 65	11 74 36 94	11 23 42 74 36 58 65 94
4	11 23 42 74	36 58 65 94	11 23 36 42 58 65 74 94

- 3 pass × 4 × 8 = **96** thao tác đọc/ghi bản ghi.

3.3 Cài đặt: phân phối và chương trình chính

```
void FileDistribute(char *Fd, char *Ft1,
                    char *Ft2, int L)
{  FILE *f  = fopen(Fd,  "rb"),
        *f1 = fopen(Ft1, "wb"),
        *f2 = fopen(Ft2, "wb");
   T a;  long k = 0;
   while (fread(&a, sizeof(T), 1, f) == 1)
   {  if ((k / L) % 2 == 0)
         fwrite(&a, sizeof(T), 1, f1);
      else
         fwrite(&a, sizeof(T), 1, f2);
      k++;
   }
   fclose(f); fclose(f1); fclose(f2);
}
```

```
void FileStraightMergeSort(char *Fd, long N)
{  for (int L = 1; L < N; L *= 2)
   {  FileDistribute(Fd, "T1.TMP", "T2.TMP", L);
      FileMerge("T1.TMP", "T2.TMP", Fd, L);
   }
   remove("T1.TMP");  remove("T2.TMP");
}
```

- **Phân phối:** phần tử thứ k thuộc run số k / L; run chẵn → Ft1, run lẻ → Ft2.
- Ví dụ L = 2: k = 0, 1 → Ft1; k = 2, 3 → Ft2; k = 4, 5 → Ft1; …
- Tập tin mở chế độ **"wb"** (ghi đè) ở mỗi pass.

Kiểm tra kết quả

Sau khi sắp, đọc lại Fd tuần tự và kiểm tra mỗi phần tử ≥ phần tử trước – cách kiểm thử đơn giản, O(N), áp dụng cho mọi thuật toán sắp xếp.

3.4 Cài đặt: trộn hai tập tin theo từng cặp run

```
void FileMerge(char *Ft1, char *Ft2, char *Fd, int L)
{  FILE *f1 = fopen(Ft1, "rb"),
        *f2 = fopen(Ft2, "rb"),
        *f  = fopen(Fd,  "wb");
   T a1, a2;
   int c1 = fread(&a1, sizeof(T), 1, f1);
   int c2 = fread(&a2, sizeof(T), 1, f2);
   while (c1 || c2)
   {  int n1 = 0, n2 = 0;   // đã lấy trong run hiện tại
      while ((c1 && n1 < L) || (c2 && n2 < L))
      {  int Con1 = c1 && n1 < L, Con2 = c2 && n2 < L;
         if (Con1 && (!Con2 || a1 <= a2))
         {  fwrite(&a1, sizeof(T), 1, f);  n1++;
            c1 = fread(&a1, sizeof(T), 1, f1);  }
         else
         {  fwrite(&a2, sizeof(T), 1, f);  n2++;
            c2 = fread(&a2, sizeof(T), 1, f2);  }
      }
   }
   fclose(f1); fclose(f2); fclose(f);
}
```

- **c1, c2:** còn dữ liệu trong Ft1, Ft2 không (fread trả về 1 hoặc 0).
- **n1, n2:** số phần tử đã lấy trong **run hiện tại** – khi n1 = L thì run của Ft1 đã hết dù tập tin còn dữ liệu.
- **Con1, Con2:** run hiện tại của mỗi tập tin còn phần tử không.
- Chọn ghi a1 khi run 1 còn và (run 2 hết **hoặc** a1 ≤ a2).
- Chỉ giữ **2 phần tử** (a1, a2) trong bộ nhớ trong ⇒ sắp xếp được tập tin lớn tùy ý.

Ví dụ 5.8a – L = 2, Ft1 = 23 42 58 65, Ft2 = 11 74 36 94

Cặp run 1: (23 42) và (11 74) ⇒ ghi 11, 23, 42 (run 1 hết, n1 = 2), 74. Cặp run 2: (58 65) và (36 94) ⇒ 36, 58, 65, 94. Fd = **11 23 42 74 36 58 65 94** ✓.

3.5 Cải tiến sắp xếp ngoại: run ban đầu và trộn nhiều đường

- **Cải tiến 1 – tạo run ban đầu bằng sắp xếp nội:** đọc từng khối **B** bản ghi vừa RAM, sắp xếp bằng QuickSort/HeapSort, ghi ra thành một run dài B ⇒ bắt đầu với L = B thay vì 1.
  - Số pass trộn giảm từ $\log_2 N$ xuống $\mathbf{\log_2(N/B)}$.
- **Cải tiến 2 – trộn tự nhiên trên tập tin:** dùng run tự nhiên như mục 2.
- **Cải tiến 3 – trộn k đường (k-way merge):** dùng k tập tin phụ, mỗi lần trộn k run cùng lúc (dùng **heap** chọn phần tử nhỏ nhất trong k phần tử đầu) ⇒ số pass **log_k(N/B)**.

Ví dụ 5.9 – Tập tin 10^8 bản ghi, RAM chứa B = 10^6 bản ghi

Phương án	Số pass trộn
Trộn trực tiếp từ L = 1	$\lceil \log_2 10^8 \rceil = 27$
Run ban đầu B = 10^6 (100 run), trộn 2 đường	$\lceil \log_2 100 \rceil = 7$
Run ban đầu B = 10^6, trộn 10 đường	$\lceil \log_{10} 100 \rceil = \mathbf{2}$

- Mỗi pass đọc + ghi toàn bộ tập tin ⇒ phương án cuối giảm vào/ra khoảng **13 lần** so với phương án đầu.

4.1 Sắp xếp theo chỉ mục – tư tưởng và cài đặt

- **Ý tưởng:** không di chuyển các bản ghi dữ liệu (lớn, nằm trên tập tin); chỉ tạo và sắp xếp một **tập tin chỉ mục** gồm các cặp **(khóa, vị trí)** nhỏ gọn. Truy xuất dữ liệu theo thứ tự bằng cách duyệt chỉ mục và dùng **fseek** đến vị trí bản ghi.

```
typedef struct { T Key; long Pos; } IdxType;
int SoSanhIdx(const void *a, const void *b)
{  T x = ((IdxType*)a)->Key, y = ((IdxType*)b)->Key;
   return (x > y) - (x < y);
}
void IndexSort(char *DataFile, char *IdxFile, long N)
{  FILE *fd = fopen(DataFile, "rb");
   IdxType *Idx = new IdxType[N];
   SinhVien sv;
   for (long k = 0; k < N; k++)
   {  fread(&sv, sizeof(SinhVien), 1, fd);
      Idx[k].Key = sv.MaSV;  Idx[k].Pos = k;
   }
   qsort(Idx, N, sizeof(IdxType), SoSanhIdx);  // sắp nội
   FILE *fi = fopen(IdxFile, "wb");
   fwrite(Idx, sizeof(IdxType), N, fi);
   fclose(fi); fclose(fd); delete[] Idx;
}
```

- **Truy xuất theo thứ tự:**

```
FILE *fi = fopen("SV.IDX", "rb"),
     *fd = fopen("SV.DAT", "rb");
IdxType e;  SinhVien sv;
while (fread(&e, sizeof(IdxType), 1, fi) == 1)
{  fseek(fd, e.Pos * sizeof(SinhVien), SEEK_SET);
   fread(&sv, sizeof(SinhVien), 1, fd);
   InSV(sv);
}
```

- Nếu chỉ mục quá lớn so với RAM ⇒ sắp xếp tập tin chỉ mục bằng **trộn trên tập tin** (mục 3).

4.2 Sắp xếp theo chỉ mục – ví dụ và đánh giá

Ví dụ 5.10 – SV.DAT (bản ghi 200 byte) và chỉ mục theo MaSV, theo DTB

Vị trí	MaSV	HoTen	DTB
0	SV07	Hoa	7.5
1	SV02	Nam	8.2
2	SV09	Châu	6.8
3	SV04	An	9.0

IDX theo MaSV	Pos
SV02	1
SV04	3
SV07	0
SV09	2

IDX theo DTB giảm	Pos
9.0	3
8.2	1
7.5	0
6.8	2

Tiêu chí	Sắp xếp trực tiếp tập tin dữ liệu	Sắp xếp theo chỉ mục
Dữ liệu di chuyển	Toàn bộ bản ghi (200 byte)	Chỉ cặp (khóa, vị trí) ≈ 12–16 byte
Nhiều thứ tự cùng lúc	Phải chọn một	Nhiều tập tin chỉ mục (theo MaSV, DTB, …)
Đọc theo thứ tự	Tuần tự – nhanh	Mỗi bản ghi một lần fseek – chậm hơn
Thêm/xóa bản ghi	Phải sắp lại	Chỉ cập nhật chỉ mục

- **Liên hệ:** đây chính là nguyên lý **chỉ mục (INDEX)** trong hệ quản trị CSDL và tìm kiếm theo chỉ mục (buổi 2).

5.1 Bảng so sánh các thuật toán sắp xếp

Thuật toán	Tốt nhất	Trung bình	Xấu nhất	Bộ nhớ phụ	Ổn định	Đặc điểm nổi bật
Interchange	$O(N^2)$	$O(N^2)$	$O(N^2)$	O(1)	Không	Đơn giản nhất
Bubble / Shaker	O(N)*	$O(N^2)$	$O(N^2)$	O(1)	Có	*có cờ dừng sớm
Selection	$O(N^2)$	$O(N^2)$	$O(N^2)$	O(1)	Không	Chỉ N − 1 hoán vị
Insertion	O(N)	$O(N^2)$	$O(N^2)$	O(1)	Có	Tốt cho dãy gần sắp, N nhỏ
Shell	O(N log N)	≈ O(N^1,25–1,5)	O(N^1,5)	O(1)	Không	Cải tiến Insertion
QuickSort	O(N log N)	O(N log N)	$O(N^2)$	O(log N)	Không	Nhanh nhất trung bình
HeapSort	O(N log N)	O(N log N)	O(N log N)	O(1)	Không	Bảo đảm N log N, tại chỗ
Merge (trực tiếp)	O(N log N)	O(N log N)	O(N log N)	O(N)	Có	Sắp xếp ngoại, DSLK
Merge (tự nhiên)	O(N)	O(N log N)	O(N log N)	O(N)	Có	Tận dụng run sẵn có

Ví dụ 5.11 – Gợi ý lựa chọn

N ≤ 50 hoặc dữ liệu gần sắp ⇒ **Insertion**; cần nhanh, không yêu cầu ổn định ⇒ **QuickSort**; cần bảo đảm thời gian xấu nhất, không thêm bộ nhớ ⇒ **HeapSort**; cần ổn định hoặc dữ liệu trên tập tin/DSLK ⇒ **MergeSort**.

5.2 Bài thực hành buổi 5

Thực hành – Đo thời gian 6–8 thuật toán với N = 10^3 đến 10^6

```
typedef void (*SortFunc)(int M[], int N);
double DoThoiGian(SortFunc f, int Goc[], int N)
{  int *M = new int[N];
   memcpy(M, Goc, N * sizeof(int));        // mọi thuật toán sắp cùng một dữ liệu
   clock_t t = clock();
   f(M, N);
   double s = (double)(clock() - t) / CLOCKS_PER_SEC;
   for (int i = 1; i < N; i++) if (M[i-1] > M[i]) printf("LOI SAP XEP!\n");
   delete[] M;
   return s;
}
```

- 1) Cài đặt StraightMergeSort, NaturalMergeSort (mảng) và gom các thuật toán buổi 3, 4 vào một chương trình.
- 2) Đo thời gian với N = 10^3, 10^4, 10^5, 10^6 trên dữ liệu ngẫu nhiên, tăng, giảm. Thuật toán $O(N^2)$ **bỏ qua** N = 10^6 nếu quá 60 giây (ghi "> 60s").
- 3) Vẽ biểu đồ thời gian theo N; kiểm chứng: N tăng 10 lần ⇒ $O(N^2)$ tăng ≈ 100 lần, $O(N \log N)$ tăng ≈ 12 lần.
- 4) (Nâng cao) Sắp xếp tập tin 10^6 số nguyên bằng FileStraightMergeSort; đếm số pass và đo thời gian.

5.3 Câu hỏi và bài tập

Câu hỏi

- 1) Trình bày thao tác trộn hai dãy có thứ tự; số phép so sánh tối đa là bao nhiêu?
- 2) So sánh trộn trực tiếp và trộn tự nhiên. Khi nào trộn tự nhiên tốt hơn rõ rệt?
- 3) Vì sao phương pháp trộn phù hợp cho sắp xếp ngoại còn QuickSort, HeapSort thì không?
- 4) Ưu, nhược điểm của sắp xếp theo chỉ mục?

Bài tập (có đáp số)

- 1) Mô phỏng trộn trực tiếp trên dãy {15, 8, 22, 3, 9, 30}. → k = 1: 8 15 · 3 22 · 9 30; k = 2: 3 8 15 22 · 9 30; k = 4: **3 8 9 15 22 30**.
- 2) Liệt kê các run tự nhiên và mô phỏng trộn tự nhiên trên dãy {3, 8, 15, 2, 9, 1, 20, 25}. → Run: [3 8 15] [2 9] [1 20 25] ⇒ lần 1: 2 3 8 9 15 · 1 20 25 ⇒ lần 2: **1 2 3 8 9 15 20 25**.
- 3) Viết hàm trộn hai tập tin số nguyên đã có thứ tự F1, F2 thành tập tin F có thứ tự.
- 4) Cài đặt sắp xếp theo chỉ mục cho tập tin SV.DAT theo **HoTen**, in danh sách SV theo thứ tự tên.